MyOwnCompiler - Day 02: Lexer

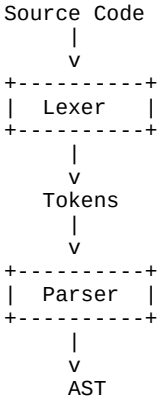
Compiler Engineering Journal

Day Objective

Build the first real compiler component: the Lexer. It converts raw source code into tokens.

What is a Lexer?

A lexer performs lexical analysis. It reads characters and groups them into meaningful tokens.



Finite State Machine (FSM)

```
         Letter
START ----------> IDENTIFIER
  |
  | Digit
  v
NUMBER

Whitespace -> Ignore
Operator   -> Emit Token
EOF        -> EndOfFile
```

Tokenization Example

```
Input:
let x = 10;

Output:
LET
IDENTIFIER(x)
EQUAL
NUMBER(10)
SEMICOLON
EOF
```

Lexer Design

```
class Lexer
{
    source;
    current;

    advance();
    peek();
    skipWhitespace();
    identifier();
    number();
    tokenize();
};
```

Complexity

Time Complexity: O(n). Space Complexity: O(n).

Interview Notes

Difference between lexer and parser. Why tokenize source code? How keywords are distinguished from identifiers? What is a finite state machine?

Revision Notes

```
Lexer: Characters -> Tokens
Parser: Tokens -> AST

Keywords:
let
if
while
print

Important methods:
advance()
peek()
skipWhitespace()
identifier()
number()
tokenize()
```

Exercises

Add +, -, *, /. Add parentheses. Add braces. Pretty-print token names. Tokenize entire files.

Day 02 Outcome

The compiler now has a lexer capable of producing token streams from source code.